

1. Executive Summary

1a. Overview

This project is an Excel-based theory crafting and resource optimisation tool modelled on the Guild Wars 2 Character build system for the purpose of achieving a desired distribution of player attributes with the use of multi-variable inputs, dynamic filtering and conditional scenario modelling. The project was designed with readability and maintainability in mind, making frequent use of named ranges, scalable source tables and helper tables to reduce formula complexity and improve long-term usability.

1b. Problem Definition: *Optimising Attributes Under System Constraints*

Guild Wars 2 does not currently offer in-game build previewing. This makes unassisted build optimisation a prolonged and often wasteful process, as the research and acquisition of the varied elements that comprise a build requires a substantial commitment of time and resources, and implementation may not yield expected results due to the limited transparency in how their respective systems interact.

1c. Solution Overview

The editor provides a unified configuration interface through which users can assemble a comprehensive build template, with inputs evaluated through a structured, modular calculation framework. Key features include:

- **Modular Bonus Logic:**
Accommodates flat, percentage-based, derived and stacking modifiers across categories.
- **Conditional Effects Simulation:**
Dedicated controls allow users to configure stacking bonuses and combat conditions.
- **Contextual UI Filtering:** Dynamic validation lists filter out irrelevant options based on user selection (e.g., Profession restrictions).

Amount	Type 1	Type 2	Conditional?
0.1	Percentage	Derived	
180	Flat		
120	Flat		
0.07	Percentage	Derived	
150	Flat		
80	Flat		
80	Flat		Yes
120	Flat		
240	Flat		Yes
120	Flat		
120	Flat		Yes
120	Flat		
120	Flat		Yes
0.02	Percentage	Stacking	Yes
0.15	Flat		Yes
0.05	Percentage		Yes
15	Flat	Stacking	Yes
0.15	Flat	Boon Enhancement	
0.04	Percentage	Stacking	Yes
0.07	Percentage		Yes
0.1	Percentage	Boon Enhancement	
0.15	Flat	Boon Enhancement	
0.1	Percentage	Derived	
150	Flat	Boon Enhancement	
0.15	Flat		

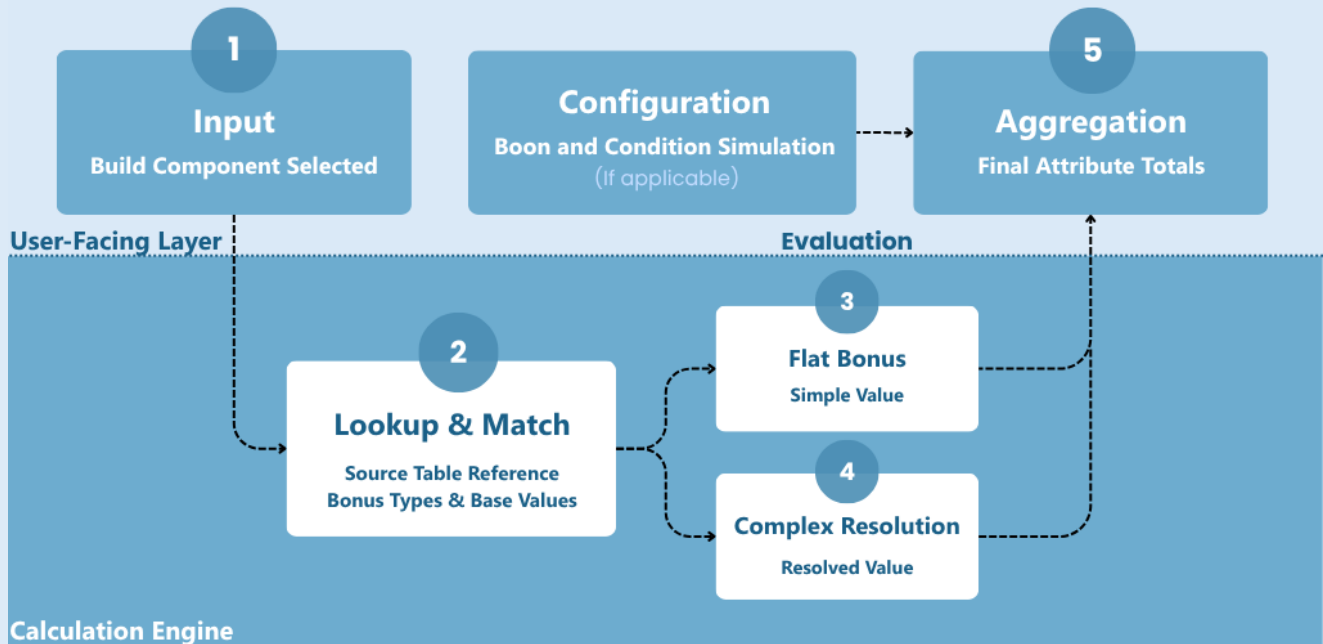
1d. Transparency and Visual Traceability

Attribute contributions from every build component are consolidated into a universal master sheet, providing a clear breakdown of cumulative totals. To ensure complete auditability, selections are dynamically highlighted in source reference tables, allowing users to trace exact base stats and formulae driving their final attribute totals.

2. Architecture and System Design

2a. System Architecture and Data Flow

Final attribute values are produced through a layered resolution process that incorporates varied calculation structures. To maintain a clean user interface, exposure to this underlying process is kept minimal – the focus of the user-facing layer is user input, resolved bonus values and final attribute values, while complex formula paths operate in dedicated background tables.



1. **User Input:** A selection is made on the main interface (e.g., Gear prefix, Utility, or Trait).
 2. **Lookup & Matching:** The selection is matched against its corresponding source table reference, which holds the base properties and valid inputs for that option category.
 3. **Flat-Bonus Components:** Simple items, like Food, return their bonus values directly to the summary engine.
 4. **Control Routing & Evaluation:** Advanced components pass metadata as control signals to dedicated resolution modules. These handle complex evaluation logic such as:
 - **Set Bonuses:** The total counts of matching Runes are evaluated with their defined threshold tiers (1-piece through 6-piece bonuses) to unlock active attribute bonus tracks.
 - **Percentage Modifiers:** Values produced by components with percentage-based bonus types are derived from a pool of attribute subtotals, which are the sum of each attribute's base and flat-bonus values, and applied multiplicatively to produce final totals.
 - **Pattern-Matched Items:** High-volume lists (like Gear) pass through a preliminary lookup table that dynamically maps bonus values based on its slot and attribute prefix rules.
 5. **Aggregation:** Resolved values are returned to the main interface, where all individual component bonuses are aggregated and added to base attributes to produce final totals.
- **Additional Configuration:** The resolved values of context-dependent components, like Traits, can be tuned in the Traits and Boons interfaces to simulate the effects of varied combat conditions.

2b. Calculation Models

Modifiers are evaluated according to their Bonus Type, ensuring every game mechanic is modelled accurately:

Modifier Type	Evaluation Mechanics	Example/Scope
Flat	Applied directly to attribute subtotal	Flat bonuses, including percentage-based attributes
Percentage	Scaled against attribute subtotal before secondary transforms	Percentage-based Trait, Utility and Signet bonuses
Derived	Percentage bonus calculated dynamically from a <i>different</i> attribute subtotal	Precision → Power % conversion
Stacking	Multiplies base value by user-configured stack count	Flat and percentage-based bonus effects of Traits and Boons
Boon Enhancement & Conditional	Gated on/off based on active UI toggle states	Simulating combat conditions

2c. Formula Optimisation and Named Range Architecture

This system makes extensive use of optimisation practices wherever practical to ensure formulas remain clean, readable, and easy to maintain or debug.

- **Named Ranges:** Key data points and reference lists essential to attribute resolution use clear, self-documenting designations in the Name Manager. This significantly improves formula legibility and makes logic errors easier to trace back to their source.
- **Error Handling & Edge Cases:** Potential nulls or incompatible selections are handled across the calculation pipeline using functions such as IFERROR, ISBLANK, and ISNA alongside default fallback values. This prevents error propagation down the line when inputs are missing or temporarily invalid.
- **Modular Logic:** Rather than relying on massive, all-in-one formulas, build resolution pipelines break complex calculations into manageable steps across dedicated lookup tables, helper columns, and intermediate processing tables.

2d. Dynamic Filtering and Conditional Formatting

Dynamic Filtering

To prevent clutter and guarantee that selections remain valid against Character Profession restrictions, all selection fields use dynamic Data Validation rules. Filter columns attached to source reference tables isolate valid options, while dedicated filtering tables group elements that depend on higher-level choices (such as organising available Traits into three distinct sets to match the three selected Specialisations).

Conditional Formatting

- **Visual Categorisation:** Green and yellow formatting styles establish clear data hierarchies, such as distinguishing primary (Major) from secondary (Minor) resolved attribute bonuses based on equipped Gear prefixes.
- **State Indicators:** Similar formatting styles call attention to active selection fields in the Traits interface, signalling when conditional or stacking traits require user tuning to take effect.
- **Compatibility Warnings:** High-contrast red fills flag selection fields that are no longer valid for the current build. This provides a real-time safety guard to offset platform limitations in Excel for the Web, which lacks native event scripts to automatically clear dependent fields when a master choice (such as Character Profession) changes.

2e. Scalability and Future Maintainability

The separation of raw reference data from dynamic processing logic ensures the workbook is fully scalable. New game updates, such as additional Specialisations, Gear prefixes or Trait lines can be integrated directly into existing source tables without requiring structural rewrites to the core calculation engine.

2f. Data Sourcing

To populate the editor's lookup tables without manual entry, custom Python ETL (Extract, Transform, Load) scrapers were built for data-heavy build components to extract, clean, and format data expediently from the Guild Wars 2 Wiki:

- **Flat Component Scrapers (e.g., Food & Utility):** Extracts item names, attribute types, and numerical values, parsing raw text into clean numeric columns.
- **Hierarchical Scrapers (e.g., Specialisations & Traits):** Executes multi-stage, paginated scrapes to capture complex dependencies—extracting Trait effects and tiers, then relationally mapping Traits to Specialisations and Specialisations to Character Professions.

Normalised CSV files produced by these scripts were then imported and integrated as source tables into the project's core framework.

3. User Interface and Useability Design

3a. Visual Hierarchy

The user-facing layer is designed with an intuitive left-to-right information sequence that broadly follows *category, selection field, effect*. These categories follow predictable visual language rules, so that even when deviations from the standard progression occur, users can quickly identify a cell's purpose (e.g., selection fields are always formatted sky blue with italic text).

3b. Clutter Reduction

With complex backend logic modules operating on separate but labelled and formatted background sheets, interacting with the core interface and tracing selections can both be achieved with minimal visual interference.

4. Iterations and Future Development

4a. Implemented Changes

Input Hierarchy and Workflow

To resolve a disjointed control flow, the left-hand control column of the primary input column was restructured into a more logical top-to-bottom functional hierarchy:

1. **Master System Controls:** Character Profession & Attribute Filter
2. **Core Mechanics & Identity:** Specialisations, Traits & Signets
3. **Primary Stat Budgets:** Gear Loadout
4. **Fine-Tuning & Secondary Modifiers:** Gear Upgrades & Consumables

This refactoring aligns dependent inputs directly beneath master controls and matches a player's natural decision-making process.

Visual Polish & Theme Consistency

Replaced inconsistent styling with a cohesive blue-and-white theme. This minimises visual fatigue during extended sessions and creates strong contrast, allowing high-priority conditional formatting alerts to stand out immediately.

4b. Future System Expansion

Secondary Component Integration

While the editor is fully functional for standard build crafting, its utility can be expanded by integrating situational systems like Infusions, relics, and underwater gear. These elements were deliberately excluded from the initial release scope as their effects are largely restricted to specialised game modes, niche combat encounters, or expansion-locked content.